

Software Design Patterns Research

Title: **Software Design Patterns Research**

Name: Junjie L. Geraldez

Course / Subject: IT342 G4

Instructor: Frederick L. Revilleza Jr.

Date: 4/6/2026

Pattern Name: Factory Pattern

Category: Creational

Problem it solves:

Creating objects manually using multiple if-else statements makes the system hard to maintain.

How it works:

A factory class is used to create objects based on input instead of directly instantiating them.

Real-world example:

Backend systems creating different services dynamically (e.g., notification services).

Use case in Parkway:

Used to create different parking types (e.g., Regular Parking, VIP Parking, Reserved Parking).

Pattern Name: Singleton Pattern

Category: Creational

Problem it solves:

Multiple instances of important services (like database connection) can cause inconsistency.

How it works:

Ensures only one instance of a class exists throughout the system.

Real-world example:

Database connection manager in backend systems.

Use case in Parkway:

Used for database connection or system configuration.

 Pattern Name: Facade Pattern

Category: Structural

Problem it solves:

Complex subsystems require multiple calls, making the code messy.

How it works:

Provides a single interface to interact with multiple subsystems.

Real-world example:

Order processing systems using one method instead of multiple service calls.

Use case in Parkway:

Used to handle parking reservation (check slot → reserve → process payment → confirm).

 Pattern Name: Adapter Pattern

Category: Structural

Problem it solves:

Different systems or APIs have incompatible interfaces.

How it works:

Wraps one interface to make it compatible with another.

Real-world example:

Integrating Google Login into a web app.

Use case in Parkway:

Used for Google Login or third-party payment APIs.

 Pattern Name: Strategy Pattern

Category: Behavioral

Problem it solves:

Multiple algorithms (like payment methods) create messy conditional logic.

How it works:

Encapsulates each algorithm and makes them interchangeable.

Real-world example:

Different payment options in e-commerce systems.

Use case in Parkway:

Used for payment methods (GCash, Card, Cash).

 Pattern Name: Observer Pattern

Category: Behavioral

Problem it solves:

When one object changes, other parts of the system are not automatically updated.

How it works:

Objects subscribe and get notified when changes occur.

Real-world example:

Live notifications or UI updates.

Use case in Parkway:

Update UI when parking slots become available or occupied.